Integration Plan: HW-554 USB Relay Board with Node.js Monorepo

This document outlines comprehensive plans and implementation strategies for integrating the HW-554 USB relay board into the existing Node.js v22 Touch Monorepo project.

1. Architecture Overview

Current System Analysis

- **Monorepo Structure:** `apps/server` (Node.js backend) + `apps/client` (React frontend)
- **Slot System:** 12 slots managed via `SlotMeta` interface in `MainPage.types.ts`
- **State Management:** Zustand stores for orders, timers, and UI state
- **Target Mapping:** Map 8 relays to slots 1-8 of existing system

Integration Goals

- **Physical Control:** Enable hardware relay control from web interface
- **State Synchronization:** Keep UI state in sync with physical relay states
- **Error Handling:** Robust error handling for hardware communication failures
- **Cross-platform:** Support Windows (client) and Linux/macOS (server)

2. Communication Method Selection

Recommended Approach: Serial Communication (Primary)

Why Serial Communication:

- **Explicit Documentation:** Clear byte commands (`0xFF, 0xFF`, `0x00, 0x00`)
- **Mature Ecosystem:** Well-established Node.js `serialport` library
- **Direct Control:** Low-level access to hardware
- **Cross-platform:** Works on Windows, Linux, macOS

Implementation:

```
// apps/server/src/services/relay.service.ts
import { SerialPort } from 'serialport';

class RelayService {
  private port: SerialPort | null = null;

  async initialize() {
    const ports = await SerialPort.list();
    const relayPort = ports.find(port =>
```

```
      port.vendorId === '0403' && port.productId === '6001' // FTDI VID/PID
    );

    if (relayPort) {
      this.port = new SerialPort({
        path: relayPort.path,
        baudRate: 9600,
        dataBits: 8,
        stopBits: 1,
        parity: 'none'
      });
    }
  }

  async toggleRelay(slotNumber: number, state: boolean) {
    if (!this.port) throw new Error('Relay board not connected');

    // Convert slot number (1-8) to relay command
    const command = this.buildRelayCommand(slotNumber, state);
    this.port.write(command);
  }

  private buildRelayCommand(slotNumber: number, state: boolean): Buffer {
    // Implementation depends on individual relay control protocol
    // For now, use simple all-on/all-off commands
    return Buffer.from(state ? [0xFF, 0xFF] : [0x00, 0x00]);
  }
}
```

Alternative Approach: CLI Wrapper (Fallback)

Why CLI Wrapper:

- **Simplicity:** Leverage existing `usbrelay` tool
- **No Native Dependencies:** Avoids build tool issues
- **Proven Solution:** Uses battle-tested external software

Implementation:

```
// apps/server/src/services/relay-cli.service.ts
import { exec } from 'child_process';
import { promisify } from 'util';

const execAsync = promisify(exec);

class RelayCLIService {
  async toggleRelay(slotNumber: number, state: boolean) {
    const command = `usbrelay ${slotNumber}=${state ? '1' : '0'}`;
    try {
      await execAsync(command);
    } catch (error) {
```

```
      throw new Error(`Relay control failed: ${error.message}`);
    }
  }
}
```

3. Server-Side Implementation

New Service Architecture

```
// apps/server/src/services/relay.service.ts
export interface RelayState {
  slotNumber: number;
  isOn: boolean;
  lastUpdated: Date;
}

export class RelayService {
  private static instance: RelayService;
  private port: SerialPort | null = null;
  private relayStates: Map<number, boolean> = new Map();

  static getInstance(): RelayService {
    if (!RelayService.instance) {
      RelayService.instance = new RelayService();
    }
    return RelayService.instance;
  }

  async initialize(): Promise<void> {
    // Initialize serial connection
    // Load saved relay states
    // Set up error handling
  }

  async toggleRelay(slotNumber: number, state: boolean): Promise<void> {
    // Validate slot number (1-8)
    // Send command to hardware
    // Update internal state
    // Emit state change event
  }

  getRelayState(slotNumber: number): boolean {
    return this.relayStates.get(slotNumber) || false;
  }

  getAllRelayStates(): RelayState[] {
    return Array.from(this.relayStates.entries()).map(([slotNumber, isOn]) => ({
      slotNumber,
      isOn,
      lastUpdated: new Date()
    }));
```

```
  }
}
```

API Endpoints

```
// apps/server/src/routes/relay/relay.routes.ts
import { Hono } from 'hono';
import { RelayService } from '../../services/relay.service';

const relayRoutes = new Hono();
const relayService = RelayService.getInstance();

// Toggle individual relay
relayRoutes.post('/toggle/:slotNumber/:state', async (c) => {
  const slotNumber = parseInt(c.req.param('slotNumber'));
  const state = c.req.param('state') === 'true';

  if (slotNumber < 1 || slotNumber > 8) {
    return c.json({ error: 'Invalid slot number' }, 400);
  }

  try {
    await relayService.toggleRelay(slotNumber, state);
    return c.json({ success: true, slotNumber, state });
  } catch (error) {
    return c.json({ error: error.message }, 500);
  }
});

// Get all relay states
relayRoutes.get('/states', async (c) => {
  const states = relayService.getAllRelayStates();
  return c.json({ states });
});

// Get specific relay state
relayRoutes.get('/state/:slotNumber', async (c) => {
  const slotNumber = parseInt(c.req.param('slotNumber'));
  const state = relayService.getRelayState(slotNumber);
  return c.json({ slotNumber, state });
});
```

4. Client-Side Integration

Enhanced SlotMeta Interface

```
// apps/client/src/pages/MainPage/MainPage.types.ts
export interface SlotMeta {
  slotType: SlotType;
  slotNumber: number;
  isChecked: boolean;
  status: SlotStatus;
  hasRelay?: boolean; // New: indicates if slot has physical relay
  relayState?: boolean; // New: current relay state
}
```

Relay Control Hook

```
// apps/client/src/hooks/useRelayControl.ts
import { useCallback } from 'react';
import { api } from 'api';

export const useRelayControl = () => {
  const toggleRelay = useCallback(async (slotNumber: number, state: boolean) => {
    try {
      const response = await api.post(`/relay/toggle/${slotNumber}/${state}`);
      return response.data;
    } catch (error) {
      console.error('Relay control failed:', error);
      throw error;
    }
  }, []);

  const getRelayStates = useCallback(async () => {
    try {
      const response = await api.get('/relay/states');
      return response.data.states;
    } catch (error) {
      console.error('Failed to get relay states:', error);
      throw error;
    }
  }, []);

  return { toggleRelay, getRelayStates };
};
```

Enhanced PadSlot Component

```
// apps/client/src/components/Pads/PadSlot/PadSlot.tsx
export const PadSlot: React.FC<PadMenuProps> = ({ slotType, slotNumber, className, variant
= 'default' }) => {
  const { mainPageSelectedSlots, toggleMainPageSlot } = useLayoutUi();
  const { toggleRelay } = useRelayControl();

  const isChecked = mainPageSelectedSlots.some(
```

```
    (selectedSlot) => selectedSlot.slotNumber === slotNumber
  );

  // Check if this slot has a physical relay (slots 1-8)
  const hasRelay = slotNumber >= 1 && slotNumber <= 8;

  const handleSelect = React.useCallback(async () => {
    // Update UI state
    toggleMainPageSlot({ slotType, slotNumber, isChecked, status });

    // If this slot has a relay, control the physical hardware
    if (hasRelay) {
      try {
        await toggleRelay(slotNumber, !isChecked);
        console.log(`Relay ${slotNumber} ${!isChecked ? 'ON' : 'OFF'}`);
      } catch (error) {
        console.error(`Failed to control relay ${slotNumber}:`, error);
        // Optionally revert UI state on hardware failure
      }
    }
  }, [slotNumber, toggleMainPageSlot, toggleRelay, hasRelay, isChecked]);

  // Rest of component...
};
```

5. State Synchronization Strategy

Real-time Updates

```
// apps/server/src/services/relay-sync.service.ts
import { WebSocket } from 'ws';

export class RelaySyncService {
  private clients: Set<WebSocket> = new Set();

  addClient(ws: WebSocket) {
    this.clients.add(ws);
  }

  removeClient(ws: WebSocket) {
    this.clients.delete(ws);
  }

  broadcastRelayState(slotNumber: number, state: boolean) {
    const message = JSON.stringify({
      type: 'relay_state_change',
      slotNumber,
      state,
      timestamp: new Date().toISOString()
    });
```

```
    this.clients.forEach(client => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(message);
      }
    });
  }
}
```

Client-Side WebSocket Integration

```
// apps/client/src/hooks/useRelayWebSocket.ts
import { useEffect, useRef } from 'react';
import { useLayoutUi } from 'providers/LayoutUiProvider';

export const useRelayWebSocket = () => {
  const wsRef = useRef<WebSocket | null>(null);
  const { updateRelayState } = useLayoutUi();

  useEffect(() => {
    const ws = new WebSocket('ws://localhost:4040/relay-ws');
    wsRef.current = ws;

    ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      if (data.type === 'relay_state_change') {
        updateRelayState(data.slotNumber, data.state);
      }
    };

    return () => {
      ws.close();
    };
  }, [updateRelayState]);
};
```

6. Error Handling and Recovery

Hardware Connection Management

```
// apps/server/src/services/relay-connection.service.ts
export class RelayConnectionService {
  private connectionState: 'disconnected' | 'connecting' | 'connected' = 'disconnected';
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 5;

  async ensureConnection(): Promise<boolean> {
    if (this.connectionState === 'connected') return true;

    try {
      this.connectionState = 'connecting';
```

```
      await this.initializeRelayBoard();
      this.connectionState = 'connected';
      this.reconnectAttempts = 0;
      return true;
    } catch (error) {
      this.connectionState = 'disconnected';
      await this.handleConnectionFailure();
      return false;
    }
  }

  private async handleConnectionFailure() {
    this.reconnectAttempts++;
    if (this.reconnectAttempts < this.maxReconnectAttempts) {
      const delay = Math.pow(2, this.reconnectAttempts) * 1000; // Exponential backoff
      setTimeout(() => this.ensureConnection(), delay);
    }
  }
}
```

Client-Side Error Handling

```
// apps/client/src/components/RelayStatus/RelayStatus.tsx
export const RelayStatus: React.FC = () => {
  const [connectionStatus, setConnectionStatus] = useState<'connected' | 'disconnected' |
'error'>('disconnected');

  useEffect(() => {
    const checkRelayStatus = async () => {
      try {
        const response = await api.get('/relay/status');
        setConnectionStatus('connected');
      } catch (error) {
        setConnectionStatus('error');
      }
    };

    const interval = setInterval(checkRelayStatus, 5000);
    return () => clearInterval(interval);
  }, []);

  return (
    <div className="relay-status">
      <span className={`status-indicator ${connectionStatus}`}>
        Relay Board: {connectionStatus}
      </span>
    </div>
  );
};
```

7. Configuration and Environment

Environment Variables

```
# .env
RELAY_ENABLED=true
RELAY_PORT=/dev/ttyUSB0  # Linux/macOS
# RELAY_PORT=COM3        # Windows
RELAY_BAUD_RATE=9600
RELAY_TIMEOUT=5000
RELAY_RECONNECT_ATTEMPTS=5
```

Configuration Service

```
// apps/server/src/config/relay.config.ts
export const relayConfig = {
  enabled: process.env.RELAY_ENABLED === 'true',
  port: process.env.RELAY_PORT || '/dev/ttyUSB0',
  baudRate: parseInt(process.env.RELAY_BAUD_RATE || '9600'),
  timeout: parseInt(process.env.RELAY_TIMEOUT || '5000'),
  maxReconnectAttempts: parseInt(process.env.RELAY_RECONNECT_ATTEMPTS || '5'),
  slotMapping: {
    // Map UI slots to physical relays
    1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 8: 8
  }
};
```

8. Testing Strategy

Unit Tests

```
// apps/server/src/services/__tests__/relay.service.test.ts
import { RelayService } from '../relay.service';

describe('RelayService', () => {
  let relayService: RelayService;

  beforeEach(() => {
    relayService = RelayService.getInstance();
  });

  it('should toggle relay state correctly', async () => {
    await relayService.toggleRelay(1, true);
    expect(relayService.getRelayState(1)).toBe(true);
  });

  it('should handle invalid slot numbers', async () => {
    await expect(relayService.toggleRelay(9, true)).rejects.toThrow('Invalid slot
number');
```

```
  });
});
```

Integration Tests

```
// apps/server/src/routes/__tests__/relay.routes.test.ts
import { testClient } from 'hono/testing';

describe('Relay API', () => {
  it('should toggle relay via API', async () => {
    const response = await testClient(app).relay.toggle.$post({
      param: { slotNumber: '1', state: 'true' }
    });

    expect(response.status).toBe(200);
    const data = await response.json();
    expect(data.success).toBe(true);
  });
});
```

9. Deployment Considerations

Docker Integration

```
# Dockerfile.relay
FROM node:22-alpine

# Install serial port tools
RUN apk add --no-cache linux-headers gcc g++ make python3

# Install usbrelay for CLI fallback
RUN apk add --no-cache usbrelay

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .
CMD ["npm", "start"]
```

Windows Build Tools

The existing `preinstall` hook for `windows-build-tools` will handle native dependencies for `serialport` and `node-hid`.

10. Implementation Timeline

Phase 1: Core Integration (Week 1)

- [ ] Install `serialport` dependency
- [ ] Create `RelayService` class
- [ ] Implement basic serial communication
- [ ] Add API endpoints for relay control

Phase 2: Client Integration (Week 2)

- [ ] Create `useRelayControl` hook
- [ ] Modify `PadSlot` component for relay control
- [ ] Add relay status indicators
- [ ] Implement error handling

Phase 3: Advanced Features (Week 3)

- [ ] Add WebSocket real-time updates
- [ ] Implement connection recovery
- [ ] Add comprehensive error handling
- [ ] Create relay status dashboard

Phase 4: Testing & Polish (Week 4)

- [ ] Write unit and integration tests
- [ ] Test on Windows client machine
- [ ] Performance optimization
- [ ] Documentation updates

11. Success Metrics

- **Hardware Control:** Successfully toggle relays 1-8 from web interface
- **State Sync:** UI state matches physical relay states
- **Error Recovery:** Automatic reconnection after hardware disconnection
- **Performance:** <100ms response time for relay commands
- **Reliability:** 99%+ uptime for relay control functionality

---

This integration plan provides a comprehensive roadmap for adding hardware relay control to the Touch Monorepo project.